

COMPARATIVO ARQUITETURAL: MONÓLITO, ARQUITETURA EM CAMADAS (MVC) E CLEAN ARCHITECTURE EM SISTEMAS ESCALÁVEIS

ARCHITECTURAL COMPARISON: MONOLITH, LAYERED ARCHITECTURE (MVC) AND CLEAN ARCHITECTURE IN SCALABLE SYSTEMS

Victor Hugo Geroto Toledo^{1, i}
Deivison Takatuⁱⁱ; Glauco Todescoⁱⁱⁱ

RESUMO

Este trabalho apresenta um estudo comparativo em desenvolvimento entre três padrões arquiteturais amplamente utilizados no desenvolvimento de sistemas de software: Monólito, Arquitetura em Camadas (MVC) e Clean Architecture. A pesquisa aborda a crescente necessidade de escolhas arquiteturais fundamentadas em evidências empíricas, considerando que a maioria das decisões técnicas nas organizações ainda se baseia em preferências subjetivas ou tendências de mercado. Para conduzir a análise, propõe-se a implementação de um protótipo tripartido — a mesma aplicação de gerenciamento de tarefas desenvolvida três vezes, cada uma seguindo um padrão arquitetural distinto — hospedado na infraestrutura em nuvem da Amazon Web Services (AWS), com coleta de métricas de desempenho por meio do Apache JMeter e monitoramento via Amazon CloudWatch e Grafana. A pesquisa encontra-se em estágio de especificação arquitetural e planejamento experimental, com a etapa de coleta de métricas ainda a ser executada. A contribuição esperada deste trabalho reside na proposição de diretrizes baseadas em evidências para a seleção de padrões arquiteturais conforme o perfil e o contexto organizacional do projeto.

Palavras-chave:

arquitetura de software; Clean Architecture; MVC; monólito; escalabilidade; computação em nuvem; AWS.

ABSTRACT

This paper presents an ongoing comparative study among three widely used architectural patterns in software systems development: Monolith, Layered Architecture (MVC), and Clean Architecture. The research addresses the growing need for architectural decisions grounded in empirical evidence, considering that most technical decisions in organizations are still based on subjective preferences or market trends. To conduct the analysis, a tripartite prototype is proposed — the same task management application developed three times, each following a distinct architectural pattern — hosted on Amazon Web Services (AWS) cloud infrastructure, with performance metrics collected via Apache JMeter and monitored through Amazon CloudWatch and Grafana. The research is currently at the architectural specification and experimental planning stage, with the metrics collection phase still to be carried out. The expected contribution of this work lies in proposing evidence-based guidelines for architectural pattern selection according to the project profile and organizational context.

Keywords:

software architecture; Clean Architecture; MVC; monolith; scalability; cloud computing;

¹ Graduando em Análise e Desenvolvimento de Sistemas — Centro Universitário SENAI São Paulo, Campus Sorocaba. E-mail: victorgeroto28@gmail.com

AWS.

1 INTRODUÇÃO

A transformação digital dos últimos anos ampliou a demanda sobre a infraestrutura e a arquitetura dos sistemas de software, que devem sustentar grande volume de solicitações simultâneas e disponibilidade constante em ambientes dinâmicos. Segundo Bass, Clements e Kazman (2012), a arquitetura de software é o conjunto de estruturas necessárias para raciocinar sobre o sistema, compreendendo seus elementos, as relações entre eles e as propriedades de ambos. A escolha do padrão arquitetural é, nesse cenário, decisão estratégica que afeta a evolução, a manutenção e a escalabilidade do sistema ao longo de todo o seu ciclo de vida (GARLAN; SHAW, 1993).

Padrões arquiteturais organizados, como a Arquitetura em Camadas — amplamente utilizada no modelo MVC (Model-View-Controller) — e a Clean Architecture, proposta por Martin (2017), oferecem métodos formalizados para a estruturação do código-fonte, favorecendo a divisão de responsabilidades, a testabilidade e a independência em relação a tecnologias externas. Estudos recentes demonstram que a escolha entre esses padrões afeta métricas objetivas como latência de resposta, complexidade ciclomática e esforço de manutenção (BLINOWSKI; OJDOWSKA; PRZYBYLEK, 2022; TAPIA et al., 2020), reforçando a importância de uma investigação empírica sistemática antes da tomada de decisões arquiteturais.

1.1 Problema de pesquisa

A dificuldade em escolher um padrão arquitetural que ofereça o melhor equilíbrio entre facilidade de implementação, manutenibilidade e escalabilidade para sistemas de software de médio porte constitui o problema central desta pesquisa. Engenheiros de software frequentemente se deparam com escolhas arquitetônicas sem o devido suporte quantitativo, fundamentando-se principalmente em preferências pessoais, modismos do mercado ou relatos anedóticos (FOWLER, 2002). Essa carência de embasamento empírico resulta em decisões subótimas que impactam diretamente a qualidade, a evolução e os custos operacionais dos sistemas.

A literatura indica vantagens e desvantagens específicas de cada padrão conforme o contexto. Blinowski, Ojdowska e Przybylek (2022) demonstraram que, em máquina única, o monólito supera os microsserviços em latência — resultado contraintuitivo que reforça a necessidade de investigação empírica rigorosa. Poucos estudos, contudo, compararam os três modelos sob as mesmas condições operacionais, o que deixa lacunas significativas para a tomada de decisão no desenvolvimento profissional.

1.2 Objetivo(s)

O objetivo principal deste estudo é analisar quantitativa e qualitativamente o desempenho, a integração e a manutenibilidade de sistemas de software criados com base nos três padrões arquiteturais mencionados, por meio da implementação e avaliação comparativa de um protótipo funcional tripartido, hospedado na infraestrutura em nuvem da AWS, com resultados exibidos em um painel administrativo de métricas em tempo real.

1.3 Justificativa

Os impactos da lacuna são expressivos: padrões inadequados produzem dívida técnica acumulada, custos crescentes de manutenção e dificuldade de escalonamento conforme a base de usuários cresce. Newman (2015) observa que a separação difusa

de responsabilidades em sistemas monolíticos transforma aplicações inicialmente simples em estruturas de difícil compreensão e evolução. A literatura disponível compara majoritariamente monólitos e microsserviços, sem considerar a Clean Architecture como alternativa estruturalmente coesa (RICHARDS, 2015).

2 REVISÃO DE LITERATURA

A arquitetura de software estabelece a organização e o desenvolvimento de sistemas por meio de decisões estruturais e princípios de design. Garlan e Shaw (1993) definem a disciplina como o estudo da organização global dos componentes de um sistema, suas interações e os princípios que governam seu projeto e evolução. Historicamente, o modelo monolítico tem sido o mais utilizado, pois combina interface, lógica de negócio e persistência de dados em um único artefato implantável, o que proporciona baixa latência em contextos de baixa complexidade, mas apresenta limitações significativas de acoplamento e escalonamento seletivo conforme o sistema cresce (DRAGONI et al., 2017).

A Arquitetura em Camadas, popularizada pelo modelo MVC, visa reduzir o acoplamento por meio da separação horizontal de responsabilidades entre camadas de apresentação, lógica de negócio e persistência, melhorando a organização do código e facilitando a escalabilidade (TU, 2023). A Clean Architecture, proposta por Robert C. Martin (2017), isola as regras de negócio no núcleo de camadas concêntricas, empregando o princípio da inversão de dependências, o que assegura alta testabilidade e independência tecnológica, reduzindo o problema frequentemente denominado “inferno de dependências” em sistemas de grande escala (DRAGONI et al., 2017).

A comparação empírica entre padrões arquiteturais constitui linha de investigação estabelecida na Engenharia de Software, mas concentrada em um recorte específico: quase todos os estudos disponíveis opõem monólitos a microsserviços, tratando a Arquitetura em Camadas e a Clean Architecture como boas práticas de estruturação interna, e não como objetos de avaliação empírica em igualdade de condições.

O trabalho de maior proximidade metodológica com esta pesquisa é o de Blinowski, Ojdowska e Przybylek (2022), publicado na IEEE Access. Os autores conduziram uma avaliação controlada de desempenho e escalabilidade entre uma aplicação monolítica e sua contraparte em microsserviços, mantendo constantes a funcionalidade e o ambiente de execução. O resultado mais relevante para o presente estudo é contraintuitivo: em ambiente de máquina única, a versão monolítica apresentou latência inferior à versão em microsserviços, atribuída à comunicação em processo, enquanto a versão distribuída demonstrou vantagem no escalonamento horizontal sob carga crescente. Esse achado sustenta a adoção do mesmo conjunto de métricas — tempo de resposta, throughput e consumo de memória — e a formulação da hipótese de que a versão monolítica apresentará menor latência em cenários de baixa carga também na comparação com as versões em Camadas e Clean Architecture.

Em direção convergente, Tapia et al. (2020) realizaram análise comparativa quantitativa entre sistemas monolíticos e de microsserviços na revista Applied Sciences, medindo consumo de recursos de hardware e latência. O procedimento adotado pelos autores — implementar a mesma funcionalidade sob arquiteturas distintas e submeter as versões a medição sob condições equivalentes — é precisamente o desenho experimental replicado neste trabalho. Em perspectiva mais ampla, Di Francesco, Lago e Malavolta (2019) conduziram estudo de mapeamento sistemático sobre arquitetura de microsserviços a partir de 103 trabalhos primários, identificando padrões de projeto, ferramentas e lacunas de pesquisa relativas a

atributos de qualidade como escalabilidade e manutenibilidade, evidenciando a predominância do eixo monólito-microserviços na produção da área.

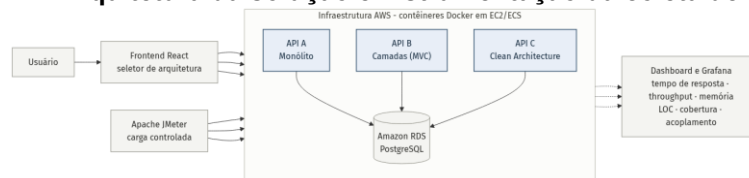
Velepucha e Flores (2023) sistematizaram, também na IEEE Access, os princípios e desafios de migração associados aos microserviços, e Abgaz et al. (2023) examinaram trinta e cinco trabalhos primários sob revisão sistemática, concluindo que a decomposição de monólitos permanece em estágio inicial de maturidade. Nenhum dos estudos identificados, contudo, avalia empiricamente a Clean Architecture em comparação controlada com o monólito e com a Arquitetura em Camadas, sob identidade funcional, de linguagem e de infraestrutura — lacuna em que se insere a contribuição deste trabalho.

3 METODOLOGIA

A metodologia adotada baseia-se na implementação de um protótipo tripartido, no qual a mesma aplicação de gerenciamento de tarefas é desenvolvida três vezes, cada versão seguindo um padrão arquitetural distinto: Monólito, Arquitetura em Camadas (MVC) e Clean Architecture. Essa abordagem permite isolar a variável independente — o padrão arquitetural — mantendo constantes o domínio de negócio, a linguagem de programação (TypeScript com Node.js), o banco de dados (PostgreSQL) e a infraestrutura de hospedagem (AWS). Cada versão contempla interface de usuário em React, camada de API REST, lógica de negócio para operações de tarefas, autenticação via JSON Web Tokens (JWT) e persistência em PostgreSQL, diferindo exclusivamente na organização estrutural do código-fonte e na distribuição de responsabilidades entre os componentes.

Cada instância do protótipo é implantada em contêineres Docker executados em instâncias Amazon EC2 de configuração idêntica, orquestrados pelo Amazon Elastic Container Service (ECS), garantindo condições homogêneas para a coleta de métricas comparativas. O armazenamento de dados é realizado por meio do Amazon RDS com motor PostgreSQL, seguindo os princípios do AWS Well-Architected Framework (AWS, 2023). Para a coleta de métricas de desempenho — tempo de resposta, throughput e consumo de memória — é utilizado o Apache JMeter na geração de carga controlada, com monitoramento centralizado por meio do Amazon CloudWatch e visualização dos indicadores em dashboards construídos no Grafana, permitindo a comparação direta dos resultados entre as três versões da aplicação, conforme ilustra a Figura 1.

Figura 1 — Arquitetura da solução e instrumentação da coleta de métricas



Fonte: elaborado pelo autor (2026).

4 RESULTADOS E DISCUSSÕES

Esta pesquisa encontra-se em estágio de desenvolvimento. Até o momento, foram concluídas a revisão de literatura, a definição da arquitetura proposta e a especificação técnica do protótipo tripartido, incluindo a escolha da pilha tecnológica (TypeScript/Node.js, React, PostgreSQL) e dos serviços de infraestrutura AWS a serem utilizados (EC2, ECS, RDS, S3, IAM e CloudWatch). A etapa de

implementação das três versões da aplicação e a consequente coleta de métricas de desempenho, por meio do Apache JMeter, do Amazon CloudWatch e do Grafana, ainda não foram concluídas, de modo que não se dispõe, neste momento, de dados quantitativos consolidados para as três arquiteturas.

Com base na fundamentação teórica revisada, formulou-se a hipótese central que orientará a análise dos dados nas próximas etapas da pesquisa: espera-se que a Clean Architecture proporcione melhores métricas de testabilidade e menor acoplamento entre módulos; que o Monólito apresente menor latência em cenários de baixa carga, replicando o resultado contraintuitivo relatado por Blinowski, Ojdowska e Przybylek (2022); e que a Arquitetura em Camadas ofereça o melhor equilíbrio entre simplicidade e organização estrutural para equipes menores. Essas expectativas serão confrontadas com os dados objetivos de desempenho, qualidade de código e manutenibilidade coletados na fase experimental subsequente, sob condições controladas que eliminam variáveis externas como divergências de domínio de negócio, linguagem de programação ou capacidade de hardware, conforme sintetiza a Figura 2.

Figura 2 — Hipóteses de comportamento relativo entre os padrões avaliados

	Monólito	Camadas (MVC)	Clean Architecture
Latência em baixa carga	● ● ● Menor	● ● ● Intermediária	● ● ● Maior
Acoplamento entre módulos	● ● ● Alto	● ● ● Intermediário	● ● ● Baixo
Testabilidade	● ● ● Baixa	● ● ● Intermediária	● ● ● Alta
Complexidade de implementação	● ● ● Baixa	● ● ● Intermediária	● ● ● Alta

Hipótese formulada a partir da literatura revisada. A verificação depende da coleta de métricas, ainda não executada.

Fonte: elaborado pelo autor (2026), a partir da literatura revisada.

5 CONCLUSÃO

Este trabalho apresentou uma proposta de estudo comparativo, em desenvolvimento, entre Monólito, Arquitetura em Camadas (MVC) e Clean Architecture, com o objetivo de fornecer diretrizes baseadas em evidências empíricas para a seleção do padrão mais adequado ao perfil e ao contexto organizacional de cada projeto, fundamentada em literatura consolidada — Martin (2017), Blinowski, Ojdowska e Przybylek (2022) e Garlan e Shaw (1993), entre outros.

O protótipo tripartido hospedado na infraestrutura da AWS possibilitará a coleta de métricas objetivas de desempenho, qualidade de código e manutenibilidade sob condições controladas, eliminando variáveis externas que comprometem a validade de comparações convencionais. As conclusões definitivas dependem da implementação das três versões e da execução dos testes de carga com o Apache JMeter. Como trabalhos futuros, propõem-se pipelines de integração e entrega contínua (CI/CD) para cada versão e a investigação da viabilidade de decomposição da versão Clean Architecture em microsserviços.

REFERÊNCIAS

- ABGAZ, Y. et al. Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review. *IEEE Transactions on Software Engineering*, v. 49, n. 8, p. 4213-4242, 2023.
- AWS. AWS Well-Architected Framework. Amazon Web Services, 2023. Disponível em: <https://aws.amazon.com/architecture/well-architected/>. Acesso em: 20 mai. 2026.

- BASS, L.; CLEMENTS, P.; KAZMAN, R. Software Architecture in Practice. 3. ed. Boston: Addison-Wesley, 2012.
- BLINOWSKI, G.; OJDOWSKA, A.; PRZYBYLEK, A. Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation. IEEE Access, v. 10, p. 20357-20374, 2022.
- DI FRANCESCO, P.; LAGO, P.; MALAVOLTA, I. Architecting with Microservices: A Systematic Mapping Study. Journal of Systems and Software, v. 150, p. 77-97, 2019.
- DRAGONI, N. et al. Microservices: Yesterday, Today, and Tomorrow. In: MAZZARA, M.; MEYER, B. (Eds.). Present and Ulterior Software Engineering. Cham: Springer, 2017. p. 195-216.
- FOWLER, M. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.
- GARLAN, D.; SHAW, M. An Introduction to Software Architecture. In: AMBRIOLA, V.; TORTORA, G. (Eds.). Advances in Software Engineering and Knowledge Engineering. Singapore: World Scientific Publishing, 1993. p. 1-39.
- MARTIN, R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Upper Saddle River: Prentice Hall, 2017.
- NEWMAN, S. Building Microservices: Designing Fine-Grained Systems. Sebastopol: O'Reilly Media, 2015.
- RICHARDS, M. Software Architecture Patterns. Sebastopol: O'Reilly Media, 2015.
- TAPIA, F. et al. From Monolithic Systems to Microservices: A Comparative Study of Performance. Applied Sciences, v. 10, n. 17, art. 5797, 2020.
- TU, Z. Research on the Application of Layered Architecture in Computer Software Development. Journal of Computing and Electronic Information Management, v. 11, n. 3, p. 34-38, 2023.
- VELEPUCHA, V.; FLORES, P. A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges. IEEE Access, v. 11, p. 88339-88358, 2023.

AGRADECIMENTOS

Agradeço primeiramente a Deus por me conceder força e determinação ao longo desta jornada acadêmica. À minha família, pelo apoio incondicional e pela compreensão nos momentos de dedicação aos estudos. Aos orientadores, Prof. Deivison Shindi Takatu e Prof. Glauco Todesco, pela orientação, paciência e contribuições fundamentais para o desenvolvimento deste trabalho. Por fim, ao SENAI, pela infraestrutura e pelo ensino de excelência que tornaram possível a realização desta pesquisa.

SOBRE O(S)AUTOR(ES)

ⁱ VICTOR HUGO GEROTO TOLEDO (Aluno)



Graduando em Análise e Desenvolvimento de Sistemas pelo Centro Universitário SENAI São Paulo — UNISENAI-SP, Campus Sorocaba, com formação técnica em Desenvolvimento de Sistemas pela mesma instituição. Atua como desenvolvedor full stack na T4E Group, com experiência em aplicações web construídas com Django, Spring Boot, Next.js e FastAPI, persistência em PostgreSQL e MongoDB e entrega contínua em nuvem. É bolsista de pesquisa e tecnologia do SESI/SENAI desde 2025, em estudo comparativo de linguagens, ecossistemas e arquiteturas de software, e possui certificação AWS Academy Graduate — Cloud Developing. Desenvolve pesquisa de

Iniciação Científica sobre padrões arquiteturais de software (Monólito, Arquitetura em Camadas e Clean Architecture) aplicados a sistemas escaláveis, com foco em avaliação empírica de desempenho e manutenibilidade em ambiente de nuvem AWS.

ii **DEIVISON SHINDI TAKATU (Orientador)**



Professor universitário, Mestre em Ciência da Computação pela Universidade Federal de São Carlos (UFSCar) Campus Sorocaba e Especialista em Informática Aplicada à Educação pelo IFSP Campus Itapetininga. Atua nas áreas de Ciência da Computação e Tecnologias Educacionais.

iii **GLAUCO TODESCO (Orientador)**



Doutor em Engenharia Elétrica pela Universidade de São Paulo (2006) e Mestre em Ciência da Computação pela Universidade Federal de São Carlos (2000). Professor de graduação em cursos de computação em Sorocaba e Itu, CEO e pesquisador da Abútua Tecnologia.